Hands-On Machine Learning

Chapter 6 — Decision Trees

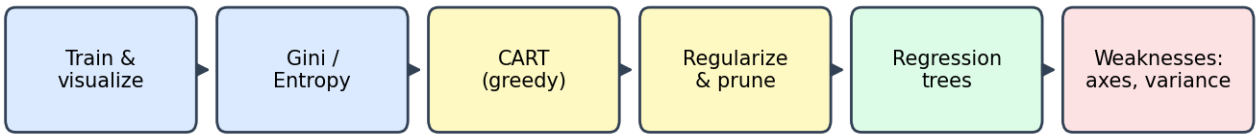
CART · Gini & Entropy · Regularization · Regression Trees

Summary of your notebook + the missing points filled in + figures and code for revision

From your notebook **Added to fill gaps** **Note / correction**

All numbers in the green **Output (real run)** boxes came from actually running the code (the real Iris data, the moons data and the regression data with the same settings as the book). The Graphviz drawing was generated with your same code (`export_graphviz` + `Source`).

Chapter map.



1 What is a Decision Tree?

From your notebook

What is a `Decision Tree`? A model that learns from a set of **sequential questions** about the `Features`, and every answer takes you to a different branch until it reaches a final decision.

→ Imagine you have `Iris` flower data and want the model to determine the flower type from the petal measurements. The tree may start with the question: is the petal length less than or equal to a certain value? (If yes go to one side, and if no move to another question.)

→ The tree can do `Classification` to determine a class or `Regression` to predict a numeric value. And it can also deal with more than one `Target` in some tasks and this is called `Multioutput`. It is also one of the basic components of `Random Forest`, which will be studied in the next chapter.

→ **The idea:** instead of having one complex equation, we have a **series of clear conditions**: a question then a branch, then another question, then a result.

Addition

(The `Multioutput` we actually tried: we trained one tree on a `target` of two columns: the flower type + whether the petal length is greater than 4.5.)

2 Training and Visualizing the tree

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
print(tree_clf.score(X_iris, y_iris), tree_clf.tree_.node_count, tree_clf.get_n_leaves())

y2 = np.c_[iris.target, iris.data["petal length (cm)"] > 4.5]          # 2 targets per flower
mo = DecisionTreeClassifier(max_depth=3, random_state=42).fit(iris.data, y2)
print(mo.predict(iris.data.iloc[[0, 100]]))
```

Output (real run)

```
0.9600 5 3      # accuracy, nodes (1 root + 1 inner + 3 leaves), leaves
[[0 0]
 [2 1]]      # shape (2, 2) for 2 flowers: [species, long_petal?] each
```

From your notebook

We will use the `Dataset` named `Iris` which has 150 flowers of 3 types. We choose only two features for training: the petal length and the petal width.

From your notebook

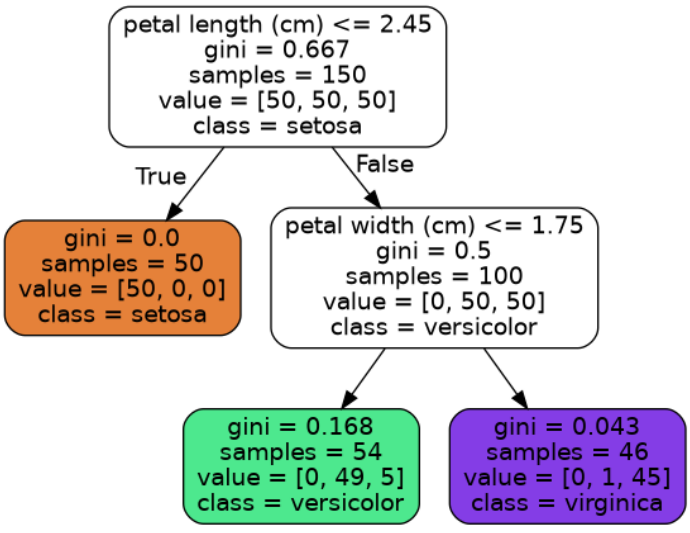
- `X_iris` : the values of the `Features` the model will use to make decisions. Each row represents a flower and the two columns are the petal length and width.
- `y_iris` : the `Target` : contains the correct type for each flower, i.e. what we want the tree to learn to predict.
- `DecisionTreeClassifier` : the tree model used in `classification` .
- `max_depth` : means the maximum depth for the tree, to keep the tree simple and easy to understand.
- `fit(X_iris, y_iris)` : is the training step: the tree searches for the suitable questions to separate the flower types.

From your notebook

Drawing the tree using Graphviz

After training we can display the questions and branches using `export_graphviz` and then open the resulting drawing:

```
from sklearn.tree import export_graphviz
from graphviz import Source
export_graphviz(tree_clf, out_file="iris_tree.dot",
                feature_names=["petal length (cm)", "petal width (cm)"],
                class_names=iris.target_names, rounded=True, filled=True)
Source.from_file("iris_tree.dot")
```



The real tree that came out of your code exactly (Graphviz): the color = the majority class in the node, and the darker it is the purer.

Output (real run)

```
digraph Tree {
node [shape=box, style="filled, rounded", color="black", fontname="helvetica"] ;
edge [fontname="helvetica"] ;
0 [label="petal length (cm) <= 2.45\ngini = 0.667\nsamples = 150\nvalue = [50, 50, 50]\nclass = setosa",
fillcolor="#ffffff"] ;
```

```
1 [label="gini = 0.0\nsamples = 50\nvalue = [50, 0, 0]\nclass = setosa", fillcolor="#e58139"] ;
0 -> 1 [labeldistance=2.5, labelangle=45, headlabel="True"] ;
...
```

From your notebook

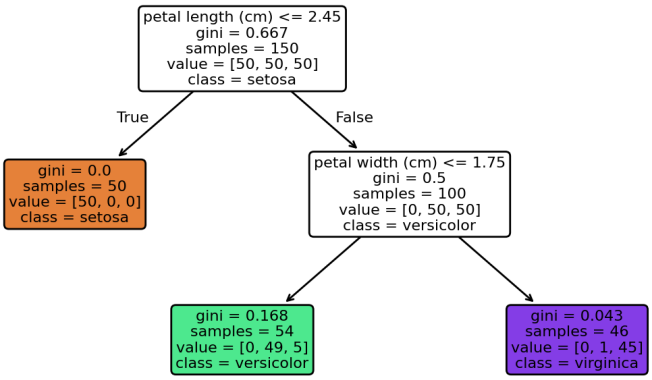
→ `export_graphviz()` writes the tree to a file in `DOT` format, and `Source.from_file()` displays the file as a drawing in an environment that supports `Graphviz`.

- **Root:** the first node in the tree.
- **Node:** a node in the tree containing a decision or test.
- **Leaf:** the end of the branch from which the prediction is made.

Addition

This table came directly from `tree_clf.tree_` (the 5 nodes in the drawing):

Node	Its question	samples	value	gini	Type
0 (root)	petal length ≤ 2.45	150	[50, 50, 50]	0.667	inner node
1	—	50	[50, 0, 0]	0.000	leaf: setosa (pure!)
2	petal width ≤ 1.75	100	[0, 50, 50]	0.500	inner node
3	—	54	[0, 49, 5]	0.168	leaf: versicolor
4	—	46	[0, 1, 45]	0.043	leaf: virginica



The same tree with `plot_tree` (matplotlib) if you do not have Graphviz: `plot_tree(tree_clf, feature_names=..., class_names=..., filled=True)`.

```
def gini(counts):
    p = np.array(counts) / sum(counts)
    return 1 - (p ** 2).sum()
print(gini([0, 49, 5]))      # the 54-sample leaf
```

```
print(gini([0, 50, 0]))      # a pure node
print(gini([50, 50, 50]))    # the root: 3 classes mixed equally
```

Output (real run)

```
0.1680
0.0000
0.6667      # = 1 - 3 x (1/3)² = 2/3, the MAXIMUM for 3 classes
```

From your notebook

- **Samples:** the number of training samples that reached the node after applying the previous conditions. For example if `samples=54` this means 54 flowers of the training data reached this node.
- **Value:** the number of samples of each `class` present in the node. If `Value = [0, 49, 5]` this means: there are no samples from the first type, and 49 from the second type, and 5 from the third type. The total 54.
- **Gini:** a measure of **how mixed the classes are** inside the node. If all the samples are from one class the `Gini` value equals zero, i.e. the node is pure (`pure`).

The Gini Impurity equation:

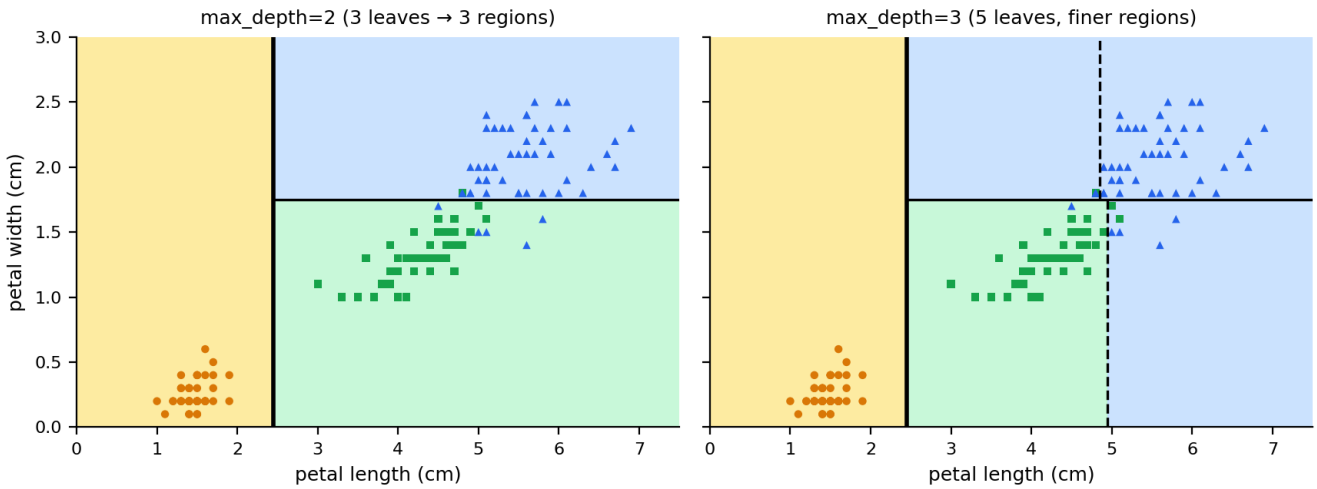
$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

From your notebook

where `p(i,k)` is the proportion of class `k` samples within node `i`.

→ The closer `Gini` gets to zero the purer the node. As for its rise, it means the classes are more mixed. The exact possible value depends on the number of classes and their distribution.

3 Decision Boundaries



The real decision boundaries for the same data: depth=2 (left): 3 regions, the thick black line is the question of the root and the thin one is the question of the second depth. depth=3 (right): 5 regions (the dashed line = a new question at depth 3) and the training accuracy 0.973 instead of 0.960.

```
scaler = StandardScaler().fit(X_iris)
tree_scaled = DecisionTreeClassifier(max_depth=2, random_state=42).fit(scaler.transform(X_iris), y_iris)
print((tree_scaled.predict(scaler.transform(X_iris)) == tree_clf.predict(X_iris)).all())
print(tree_clf.tree_.threshold[0], "->", tree_scaled.tree_.threshold[0] * scaler.scale_[0] +
scaler.mean_[0])
```

Note / Correction

In your notebook you wrote that the `Gini` of the node `[0, 49, 5]` "is approximately 0.268". The correct calculation: `1 − (49/54)² − (5/54)² = 0.168` (and this is the same number in the Graphviz drawing above: `gini = 0.168`). It seems you flipped the 1 with the 2 while writing.

And by the way: the maximum value of `Gini` with 3 classes is 2/3 (0.667) and not 1, and with two classes 0.5.

From your notebook

The tree does not only learn the questions, but it also **divides the data space into regions**, and since we use two features we can draw these regions on a 2D plane.

→ With `max_depth=2` the tree is limited in depth, so it cannot make many splits. If we allowed it a bigger depth like `max_depth=3` it can add other conditions and divide the regions into smaller parts, and thus **the decision boundaries change**.

From your notebook

→ **The advantage of trees here:** they do **not** need normalization or standardization of the `Feature` values, because each test like `petal length ≤ 2.45` does not depend on the distance between the values or on being on the same scale.

Output (real run)

```
True
2.45 -> 2.45      # same split, just expressed in scaled units
```

4 `tree_` and White Box vs Black Box

```
from sklearn.tree import export_text
print(export_text(tree_clf, feature_names=["petal length (cm)", "petal width (cm)"]))
t = tree_clf.tree_
print(t.node_count, t.max_depth, t.children_left, t.children_right, t.feature)
```

Output (real run)

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2
5 2 [1, -1, 3, -1, -1] [2, -1, 4, -1, -1] [0, -2, 1, -2, -2]
```

Addition

A real experiment to confirm your note: I compared the tree on the original data and the tree on the data after `StandardScaler`: **exactly the same predictions** and the same split threshold (2.45 after I converted it back to its units). Unlike the SVM we saw in Chapter 5 (where scaling is necessary).

From your notebook

After training a tree scikit-learn lets you access information about its structure through the attribute `tree_` (`tree_clf.tree_`), which contains internal information about the **nodes and the splits**, and it can be used when examining how the model works.

→ The tree is usually considered a `White Box Model` because we can display the questions and branches that led to the decision and read them. In contrast there are models closer to a `Black Box` where interpreting the decision from inside the model is much harder.

→ But the ease of reading the tree **does not alone guarantee** that its decisions are correct or fair; it only means that the decision steps are visible and inspectable.

Addition

`export_text` prints the tree as `if/else` rules: exactly the meaning of "White Box": you can read the decision in text and explain it to a non-technical person. And `-1` in `children_left` means "this is a leaf, it has no children".

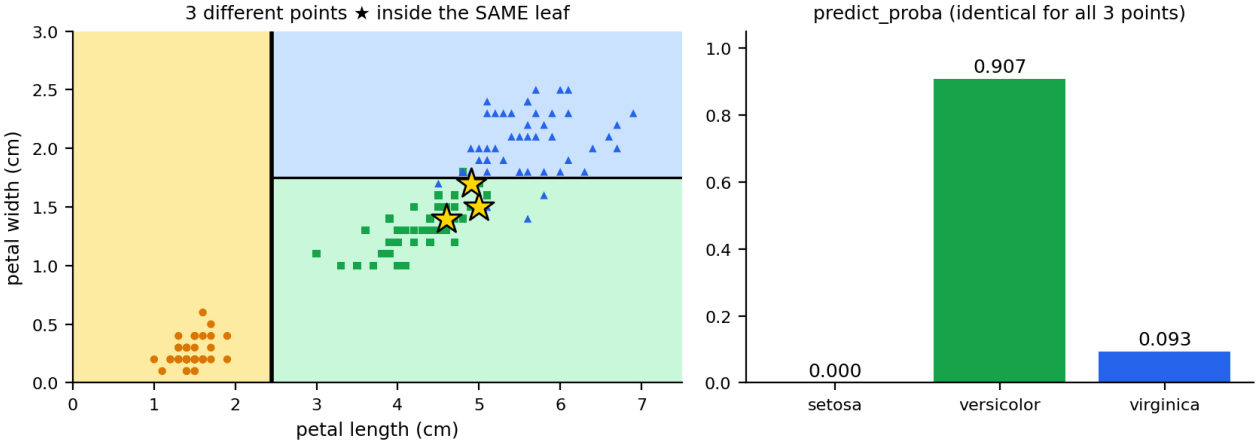
And for interpretation also: `feature_importances_` (the sum of the `impurity` decrease each feature made). On Iris with four features and depth 3: the tree depended on the petal measurements only: the importance of the sepals = exactly 0 (not used in any question), and petal length = 0.586 and petal width = 0.414. This is exactly what we saw in the drawing.

Output (real run)

```
sepal length (cm) 0.0
sepal width (cm) 0.0
petal length (cm) 0.586
petal width (cm) 0.414
```

5 Estimating Class Probabilities

```
print(tree_clf.predict_proba([[5, 1.5]]))
print(tree_clf.predict([[5, 1.5]]))
```



A real experiment: 3 different points (the stars) inside the same leaf get identical probabilities [0.0, 0.907, 0.093]. See `[4.6, 1.4]` = [0.0, 0.907, 0.093] and `[4.9, 1.7]` = [0.0, 0.907, 0.093].

From your notebook

The tree can not only predict the flower type, it can also **estimate the probability** of belonging to each class using `predict_proba()`:

Output (real run)

```
[[0.000 0.907 0.093]]
[1]
```

From your notebook

The input `[5, 1.5]` represents a flower with petal length 5 cm and width 1.5. The tree follows the conditions until it reaches a leaf containing **54 training samples**: 49 of the second type and 5 of the third type and none of the first type. So the probabilities are computed from the **proportions inside the leaf**:

- `P(Setosa) = 0/54 = 0`
- `P(Versicolor) = 49/54 ≈ 0.907`
- `P(Virginica) = 5/54 ≈ 0.093`

Therefore the prediction is `Class 1`, i.e. Versicolor, because it has the highest probability.

An important point: any point falling in the **same leaf** gets **the same probabilities** in this tree, even if the values of its `Features` differ, as long as it reached the same leaf.

Addition

This means that the tree's probabilities are **coarse**: there is no "gradation" inside the leaf (a point close to the boundary is like a point in the middle of the leaf). They are also probabilities built on a small number of samples (54), so do not depend on them as you would depend on the probabilities of a smooth model like Logistic Regression.

6 The CART algorithm (how does the tree learn?)

```
def split_cost(X, y, k, t):
    left, right = X[:, k] <= t, X[:, k] > t
    g = lambda yy: 1 - sum((yy == c).mean() ** 2 for c in np.unique(y))
    return left.mean() * g(y[left]) + right.mean() * g(y[right])

for k, name in enumerate(["petal length", "petal width"]):
    values = np.unique(X_iris[:, k]); thresholds = (values[:-1] + values[1:]) / 2     # try EVERY
midpoint
    J = [split_cost(X_iris, y_iris, k, t) for t in thresholds]
    print(name, "best threshold =", round(thresholds[np.argmin(J)], 2), " J =", round(min(J), 3))
```

Output (real run)

```
petal length best threshold = 2.45  J = 0.333
petal width best threshold = 0.80  J = 0.333
```

From your notebook

Scikit-learn uses an algorithm called **CART** (`Classification and Regression Tree`) to build the tree, and the basic idea is that at every node it chooses a `Feature` and a `Threshold` to split the data into two groups.

→ For example it may compare many possible questions: is the petal length less than 2.45 cm? is it less than 5? ... The algorithm chooses the question that **reduces the impurity of the two resulting groups in the best way** according to the cost function.

→ **The splitting cost function in Classification:**

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

From your notebook

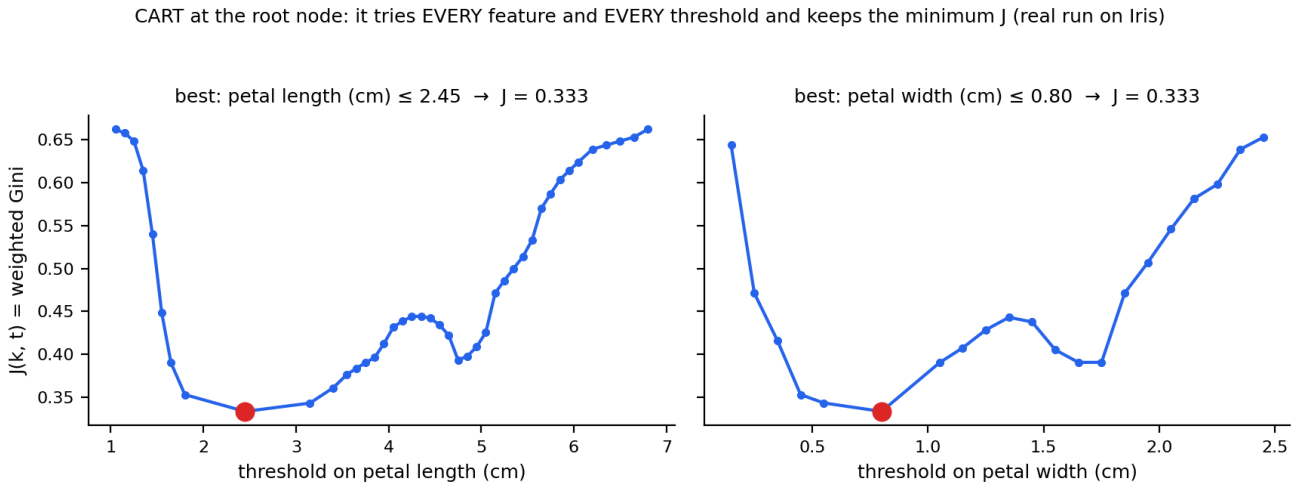
- `k` : the `Feature` you choose.
- `t_k` : the `Threshold` we will split at.
- `m` : the number of samples in the current node, `m_left` , `m_right` : the number of samples in the left and right branches,
- `G_left` , `G_right` : the impurity values in the two branches.

We compute **the average impurity weighted by the number of samples**, so if a branch had 100 samples and the other 5 samples, the bigger branch will have a bigger weight in the calculation.

Addition

A numeric example from you (100 and 5 samples): if the big branch has `gini = 0.5` and the small one 0 (pure): `J = 100/105 × 0.5 + 5/105 × 0 =` **0.476**. That is, the small branch only affects 5% of the decision.

And on the root of Iris: the left branch (50, setosa) `gini = 0` and the right (100) `gini = 0.5` → `J = 50/150 × 0 + 100/150 × 0.5 =` **0.333** (and this is the best J the algorithm found).



This is exactly what CART does at the root: it tries every feature and every threshold and takes the lowest J. (a real run on Iris)

Addition

An important discovery from the experiment: the two features produced the **same lowest J** (0.333)! That is, there is a **tie** between `petal length ≤ 2.45` and `petal width ≤ 0.8` (both separate setosa perfectly). And sklearn breaks the tie **randomly** (according to `random_state`), and this is the reason why different trees appear in section 13.

7 Computational Complexity

From your notebook

Why is it called a Binary Tree?

In CART every split produces **only two branches** (always). That is, even if we had three classes or ten classes, every question in the tree gives two branches. Some other tree algorithms such as `ID3` in their traditional form can produce more than one branch from a single node according to the values of the `Feature` used in the split.

From your notebook

Greedy Algorithm

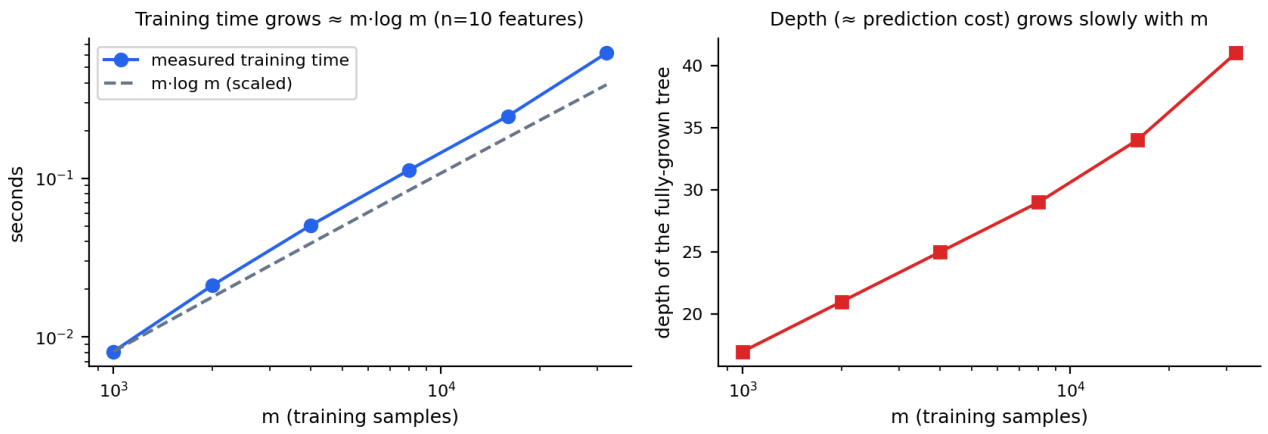
CART learns in a `Greedy` way: it chooses the **best split for the current node** according to the criterion each time, then repeats the process on the resulting nodes, but it does **not** try all the possible tree shapes in order to guarantee getting the best final tree at the level of all the data.

Finding the globally optimal tree is a computationally very hard problem; the book explains that finding the best decision tree is considered `NP-complete` (the solution time grows with the problem size as `O(exp(m))`) and therefore a `Greedy` algorithm is used as a reasonable solution instead of exhaustive search.

CART continues splitting the branches until a stopping condition is satisfied such as reaching `max_depth` or the absence of a suitable split that lowers the cost function according to the settings used.

From your notebook

If we have `m` training samples and the tree is roughly balanced, then the prediction process usually requires passing through a number of nodes proportional to the **depth of the tree**, i.e. roughly `O(log₂(m))` in the case of a balanced tree.



A real experiment (n=10 features, data with noise): left: the training time really goes with m·log m roughly (the dashed line). Right: the tree depth (= the prediction cost).

From your notebook

→ If the tree is **badly unbalanced** prediction may become slower, because the complexity depends on the number of levels the actual path passes through and not on the number of samples alone.

→ As for **training** the tree using the search for the best `Threshold` for each `Feature` at every node, the approximate complexity the book mentions is `O(n × m log m)` (`n` = the number of Features, `m` = the number of samples). The formulation is not always perfectly linear because of the `log m` factors.

Note / Correction

From the real measurement: when `m` grew from 1,000 to 32,000 the tree depth became **41** although `log₂(32,000) ≈ 15`. The reason is that the tree that grows without restrictions on data with noise is **not balanced** (as you said in your notebook: the complexity depends on the actual depth). And despite that, prediction is very fast: 59 microseconds per sample after training on 32,000 samples.

8 Gini Impurity vs Entropy

From your notebook

Until now we used `Gini` but there is another common way called `Entropy`; it measures the **uncertainty** inside the node.

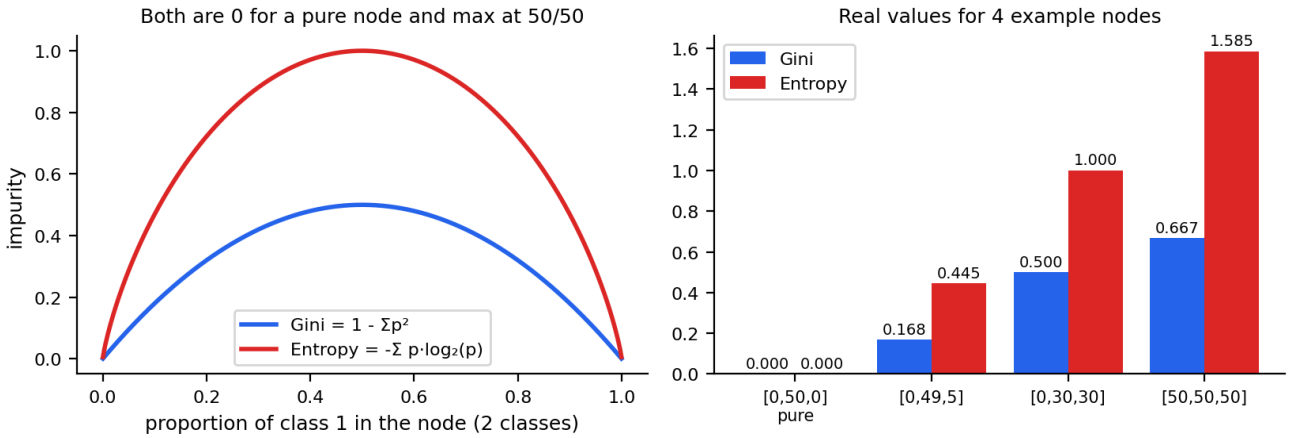
$$H_i = -\sum_{k=1,\ p_{i,k}>0}^{n} p_{i,k} \log_2(p_{i,k})$$

From your notebook

→ In the equation `p(i,k)` is the proportion of class `k` in node `i`. If the node contains only one class the `Entropy` becomes equal to zero because there is no uncertainty in it.

→ For example: in a node whose samples total 54 `[0, 49, 5]` `Entropy ≈ 0.445` while the `Gini` for this node ≈ `0.168`.

```
def entropy(counts):
    p = np.array(counts) / sum(counts); p = p[p > 0]
    return -(p * np.log2(p)).sum()
print(entropy([0, 49, 5]), entropy([50, 50, 50]))
tree_ent = DecisionTreeClassifier(criterion="entropy", max_depth=2, random_state=42)
```



Left: both start from 0 (pure) and reach their maximum at 50/50; the entropy is higher and its curve is sharper. Right: real values for 4 nodes.

Output (real run)

```
0.4451 1.5850      # entropy of [0,49,5] and of the root (log2(3) = 1.585)
```

From your notebook

→ In scikit-learn `DecisionTreeClassifier` uses the `Gini` criterion by default, and if we wanted to use `Entropy` we can write `criterion="entropy"`.

→ The two criteria often lead to **similar trees**, but the shape of the resulting tree may differ a bit. The `Gini` calculation is usually simpler and faster.

Likewise `Gini` tends in some cases to isolate the most frequent class inside a branch, while `Entropy` may produce somewhat more balanced splits. This is a general tendency and not an absolute rule in every `Dataset`.

Addition

A real experiment:

1. On Iris with depth 3 both criteria gave **exactly the same tree** (the same rules) and accuracy 0.973.
2. But on 40 different `moons` datasets the trees came out different in **36 of the 40** cases (at the same depth). So "similar" in performance, not in shape.
3. Speed: I measured training a tree on 20,000 samples × 20 features: `gini` = 0.58 s vs `entropy` = 0.46 s; i.e. I did not find a consistent difference in speed in my experiment (the difference is within the noise), so the best is to choose by validation, not by speed.

From your notebook

The decision tree is a **Nonparametric** model in the usual sense (we do not have a fixed number of model parameters that we determine before training, but rather the shape and size of the tree is determined by the data itself).

9 Regularization (reducing Overfitting)

```
from sklearn.datasets import make_moons
from sklearn.metrics import accuracy_score
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
tree_unregularized = DecisionTreeClassifier(random_state=42)
tree_regularized = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_unregularized.fit(X_moons, y_moons)
tree_regularized.fit(X_moons, y_moons)
X_test, y_test = make_moons(n_samples=1000, noise=0.2, random_state=43)
pred_unregularized = tree_unregularized.predict(X_test)
pred_regularized = tree_regularized.predict(X_test)
print(accuracy_score(y_test, pred_unregularized))
print(accuracy_score(y_test, pred_regularized))
```

From your notebook

→ **The advantage:** it can represent complex patterns.

The problem: if the tree is left without restrictions it may learn small details and noise specific to the training data, then it can achieve excellent performance on the training data but its performance is weaker on new data. That is the `overfitting` problem.

→ **The most important Hyperparameters:**

1. `max_depth` : sets the maximum depth of the tree; reducing the value limits its complexity.
2. `min_samples_split` : sets the minimum number of samples a node must contain so it is allowed to try splitting it.
3. `min_samples_leaf` : sets the minimum number of samples that must remain in each leaf after the split; raising this number prevents creating very small leaves.
4. `min_weight_fraction_leaf` : sets the minimum weight of the samples in the leaf as a fraction of the total sample weights.
5. `max_leaf_nodes` : sets the maximum number of leaves in the tree.
6. `max_features` : sets the number of `Features` taken into consideration when searching for the best split in the node, according to the specified value.

The general rule: increasing restrictions such as reducing `max_depth` or `max_leaf_nodes` or raising `min_samples_split` / `min_samples_leaf` makes the tree simpler and often helps reduce `overfitting` , but increasing the restrictions more than necessary may cause `Underfitting` .

From your notebook

The book's example: Moons:

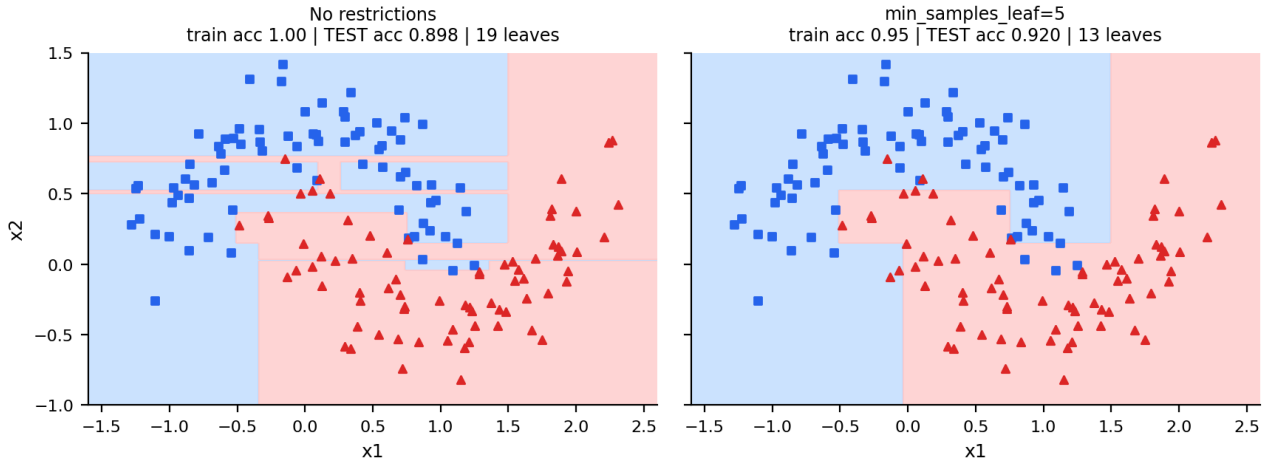
Output (real run)

```
0.898
0.920
```

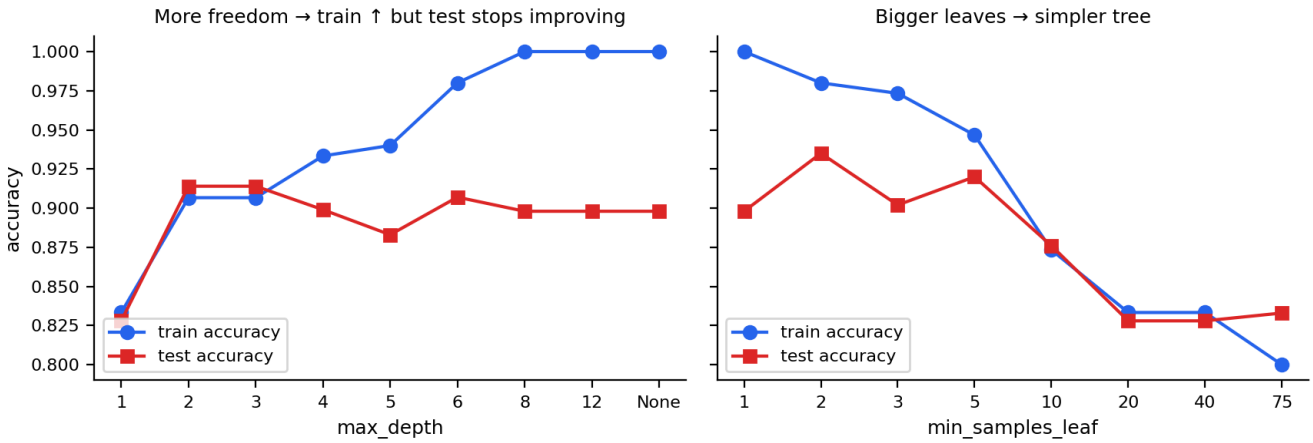
From your notebook

→ In the example: the unrestricted tree achieved `Accuracy` of `0.898` while the tree that uses `min_samples_leaf=5` achieved `0.92` on the test set used in the book. (And the real run gave exactly the same two numbers.)

→ **The point:** it is not that adding restrictions will always raise the accuracy, but the first tree can create many branches and leaves to fit the details of the training data, whereas the restriction prevents some of these splits and makes the model more able to generalize here.



A real experiment on the same data: left: the unrestricted tree (19 leaves, training accuracy 1.00) makes small rectangles around isolated points. The restricted one (13 leaves) is smoother and its test accuracy is higher.



Left: whenever we allowed a larger depth the training accuracy went up to 1.00 but the test accuracy stopped (and went down and back up) after depth 2-3. Right: `min_samples_leaf`: larger = a simpler tree, but if we increased it more than necessary (+20) both go down (underfitting).

Addition

Notice on the right: the best test accuracy came out at `min_samples_leaf=2` (0.935) and not at 5 (0.920), and with 75 it goes down to 0.833 (the tree became 2 leaves only). So **there is no magic value**: we choose it with validation (like `GridSearchCV` as we learned).

10 Pruning the tree

From your notebook

Pruning: another way to reduce complexity is to let the tree **grow first**, then we **delete the branches** that do not give an improvement that is statistically significant in the purity of the nodes. This is the principle of `Pruning`.

→ The book mentions using a test like `χ²` to estimate whether the apparent improvement in the split may be **just chance**. The `p-value` is used to judge that, and if its value is greater than a certain limit (like `5%` in the example the book mentions) the improvement may be considered insufficient and the node and its children are deleted, and the process repeats until no unnecessary nodes remain according to the test.

→ So `Pruning` is another method to control tree complexity besides controlling the `Hyperparameters` during training.

Addition

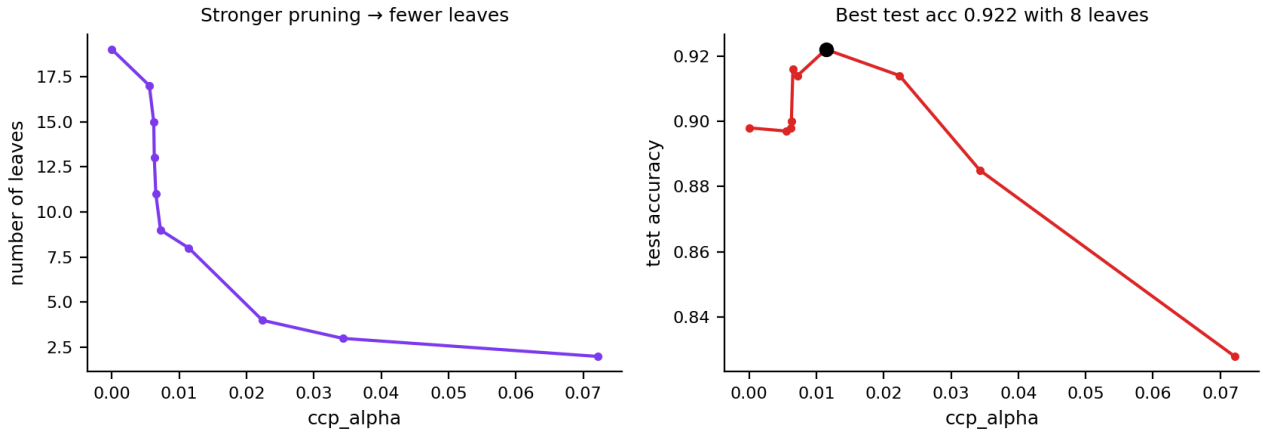
Note: scikit-learn does **not** implement the `χ²` test of the book. What exists in it is another form of pruning called **Cost-Complexity Pruning**: `α`: we put a penalty on the number of leaves, and we delete the branches whose improvement is less than the penalty:

$$R_\alpha(T) = R(T) + \alpha\,|\tilde{T}| \qquad (|\tilde{T}| = \text{number of leaves})$$

```
path = DecisionTreeClassifier(random_state=42).cost_complexity_pruning_path(X_moons, y_moons)
for alpha in path.ccp_alphas[:-1]:
    t = DecisionTreeClassifier(random_state=42, ccp_alpha=alpha).fit(X_moons, y_moons)
    print(round(alpha, 4), t.get_n_leaves(), round(accuracy_score(y_test, t.predict(X_test)), 3))
```

Output (real run)

```
0.0000 19 0.898
0.0062 15 0.898
0.0065 11 0.916
0.0114  8 0.922
0.0343  3 0.885
...
```



A real experiment: every time we raised `ccp_alpha` the leaves decrease, and the test accuracy improves up to a point (the black point) and then goes down afterwards (the increase of pruning = underfitting).

Addition

(We read: `ccp_alpha`, the number of leaves, the test accuracy.) **Best result:** at only 8 leaves accuracy **0.922** (instead of 19 leaves in the full tree with accuracy 0.898). And this is the same idea as yours: pruning the weak branches after growth.

11 Decision Tree Regression

From your notebook

Until this point we used the tree for **classification** only, but it can also be used in `Regression` to predict a continuous numeric value via `DecisionTreeRegressor`.

```
from sklearn.tree import DecisionTreeRegressor
X_quad = np.random.rand(200, 1) - 0.5
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
print(tree_reg.predict([[0.2]]))
```

Output (real run)

```
[0.0277]      # R² on the training data = 0.797
```

From your notebook

→ Here `X_quad` and `y_quad` are the `Regression` data that the book example created. The variable `max_depth=2` restricts the depth of the tree as we did in classification, but **the basic difference** is the way the split quality and the result produced by the leaf are evaluated.

→ In `Regression`, every leaf produces **the mean of the `Target` values** of the training samples that reached it, i.e. the tree does **not** choose a `class` name as happens in `Classification`; rather it produces a **numeric value**.

→ **The cost function in Regression:** instead of reducing `Gini` or `Entropy`, the split tries to reduce the **weighted mean squared error (MSE)** in the two resulting branches.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}}$$

$$\text{MSE}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} (y^{(i)} - \hat{y}_{\text{node}})^2 , \qquad \hat{y}_{\text{node}} = \frac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)}$$

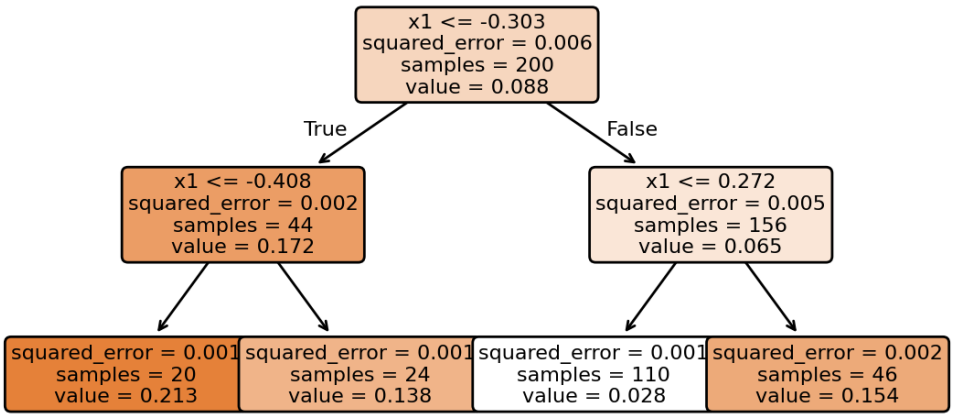
where `ŷ_node` is the mean of the `Target` inside the node and `y(i)` is the actual value.

Addition

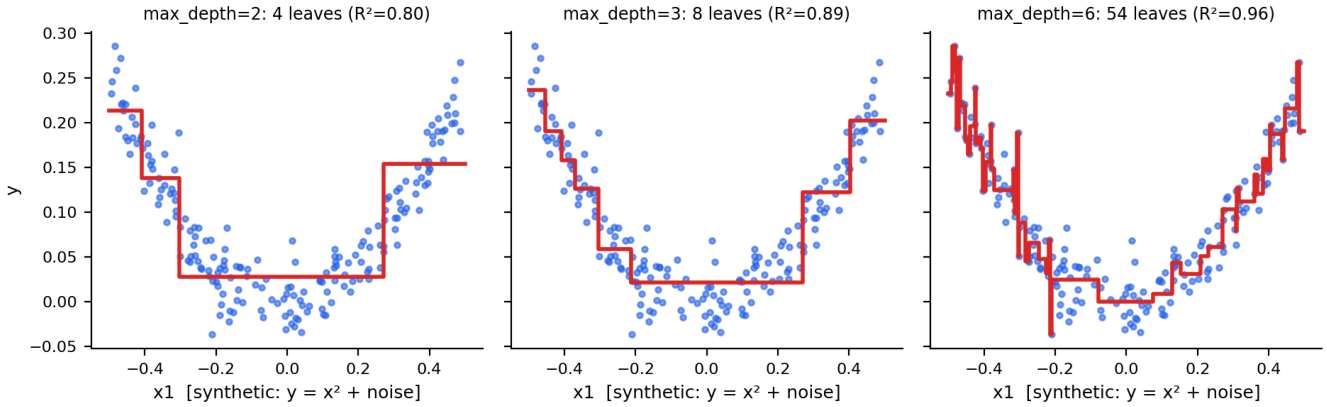
With real numbers: the root split on `x1 ≤ -0.303` (200 samples, `MSE = 0.0061`, mean target = 0.088). The left branch 44 samples (`MSE` 0.0024) and the right 156 samples (`MSE` 0.0046). The cost `J = 44/200 × 0.0024 + 156/200 × 0.0046 =` **0.00413**. And this is the same `squared_error` that you see in the drawing below (it is called `squared_error` instead of `gini`).

Output (real run)

```
|--- x1 <= -0.30
|   |--- x1 <= -0.41
|   |   |--- value: [0.21]
|   |--- x1 >  -0.41
|   |   |--- value: [0.14]
|--- x1 >  -0.30
|   |--- x1 <= 0.27
|   |   |--- value: [0.03]
|   |--- x1 >  0.27
|   |   |--- value: [0.15]
```



The real regression tree: every leaf has `value` = the mean of the target for the samples that reached it (this is the prediction).



The tree prediction = a step function (a constant value inside each leaf). depth=2: 4 leaves (R^2=0.80), depth=3: 8 leaves (R^2=0.89), depth=6: 54 leaves (R^2=0.96) but it follows the noise (overfitting).

Addition

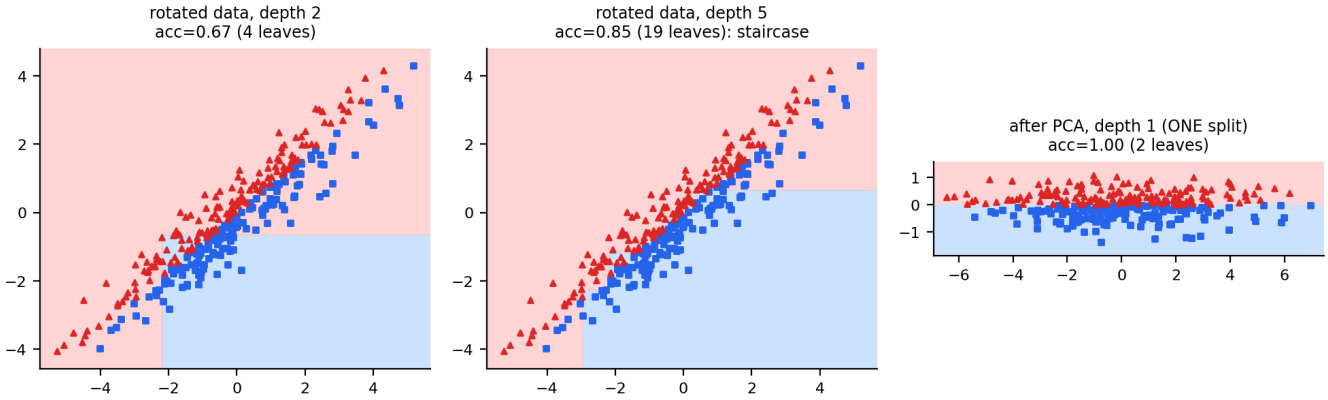
The same regularization principle in section 9 applies here: `min_samples_leaf` and `max_depth` ... And also the `Regression tree` **does not do extrapolation**: outside the data range (for example `x=2`) it returns the value of the last leaf (constant), unlike `Linear Regression`, which continues the line.

12 Sensitivity to Axis Orientation

From your notebook

Among the important weaknesses of `Decision Tree` is that it tends to build **decision boundaries perpendicular to the axes**, i.e. vertical and horizontal splits when working with two features. This may make separating the data harder.

→ Imagine you have data that is easy to separate with **one slanted line**. The tree may need **several vertical and horizontal splits** in order to approximate the slanted line, instead of using one simple boundary.



A real experiment: data (synthetic) is separated by one slanted line. Left: a depth=2 tree: its accuracy is only 0.67. Middle: depth=5 (19 leaves) reaches 0.85 by means of a "staircase". Right: after PCA (rotating the axes): one single question separates them (2 leaves) 1.00.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
pca_pipeline = make_pipeline(StandardScaler(), PCA(n_components=2))
X_iris_pca = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_pca, y_iris)
print(tree_clf_pca.score(X_iris_pca, y_iris), pca_pipeline[-1].explained_variance_ratio_)
print("root split on component", tree_clf_pca.tree_.feature[0] + 1, "<=",
      round(tree_clf_pca.tree_.threshold[0], 2))
```

From your notebook

Does PCA help?

`PCA` can help by changing the data representation and the direction of its axes. When we apply `PCA` on the features we get **new components** such as `z1` and `z2`. These components represent **new directions** in the data space.

→ In the book's example after preparing the data and using `PCA` the tree was able to separate the classes using **one** `feature` from the new components which is `z1`. But one must pay attention that `PCA` **often** helps, **not always**: it is not guaranteed that improving the direction of the data will lead to a better tree in every `Dataset`.

Output (real run)

```
0.9800 [0.9814 0.0186]
root split on component 1 <= -0.91
```

From your notebook

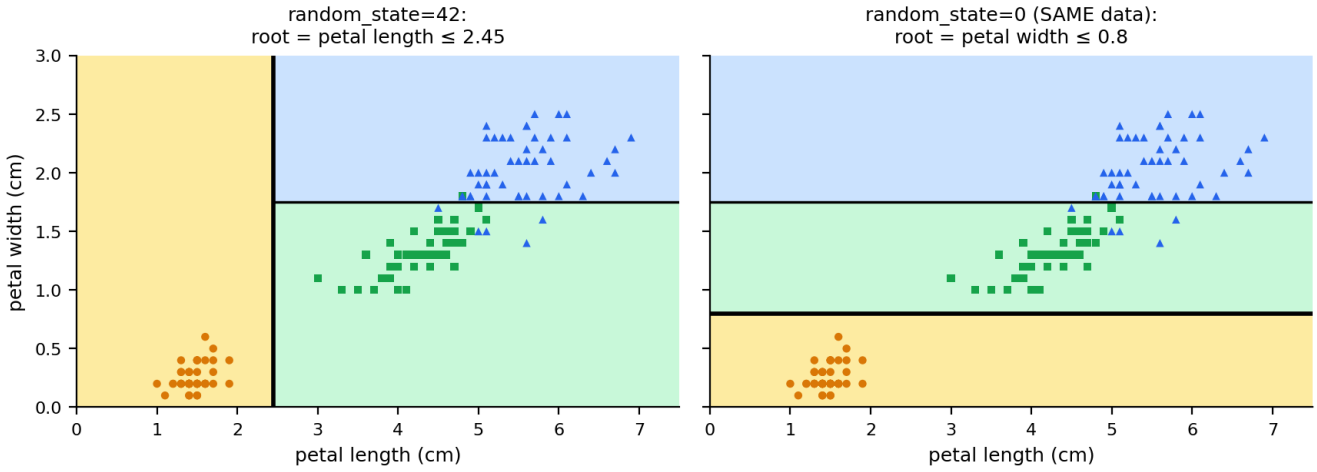
→ We used `StandardScaler` before `PCA` because `PCA` is affected by the preparation of the features before computing the components, and then trained a tree on the data after transforming it. This is an example showing the effect of the new data representation, and it does not mean that every `Decision Tree` needs `PCA` or `Scaling` in order to work.

Addition

Here it is useful to link the two remarks: the tree itself does not need scaling (section 3), but the `PCA` before it is what needs it.

On the real Iris: the tree's accuracy after PCA = **0.980** versus **0.960** on the original features. So there is no real improvement here (the data is separable with roughly perpendicular boundaries to begin with), and this confirms your warning "often, not always". The first component alone holds 98% of the variance.

13 The Decision Tree has High Variance



A real experiment on exactly the same data, only by changing `random_state` (left 42, right 0): the root changed from petal length to petal width and the decision boundaries changed completely.

Addition

I tried 30 `random_state` values: 16 chose `petal width` and 14 chose `petal length`. (This is the tie we talked about in section 6: both give the same J.)

And there is another explicit experiment: I removed one flower (versicolor at 4.8 × 1.8, which the book uses as an example): in a tree with `max_depth=2` no change appeared (`True`: the same tree), but in an **unrestricted** tree on the petal features the number of leaves changed from 8 to 7 and the rules differed (`False`). That is, the variance appears more the larger the tree is, and this is exactly what brings us back to regularization (section 9).

From your notebook

What is meant here is that the decision tree **may change noticeably** when small changes occur in the training data or when there is a difference in the choices that affect building the tree. The order of the questions and the branches may change, and thus the decision boundaries change.

→ This explains why you may get **different trees although the data is very similar**. `random_state` can be used to make training reproducible under the same conditions, but it **does not make the algorithm stable**: the tree's shape changes when the training data actually changes.

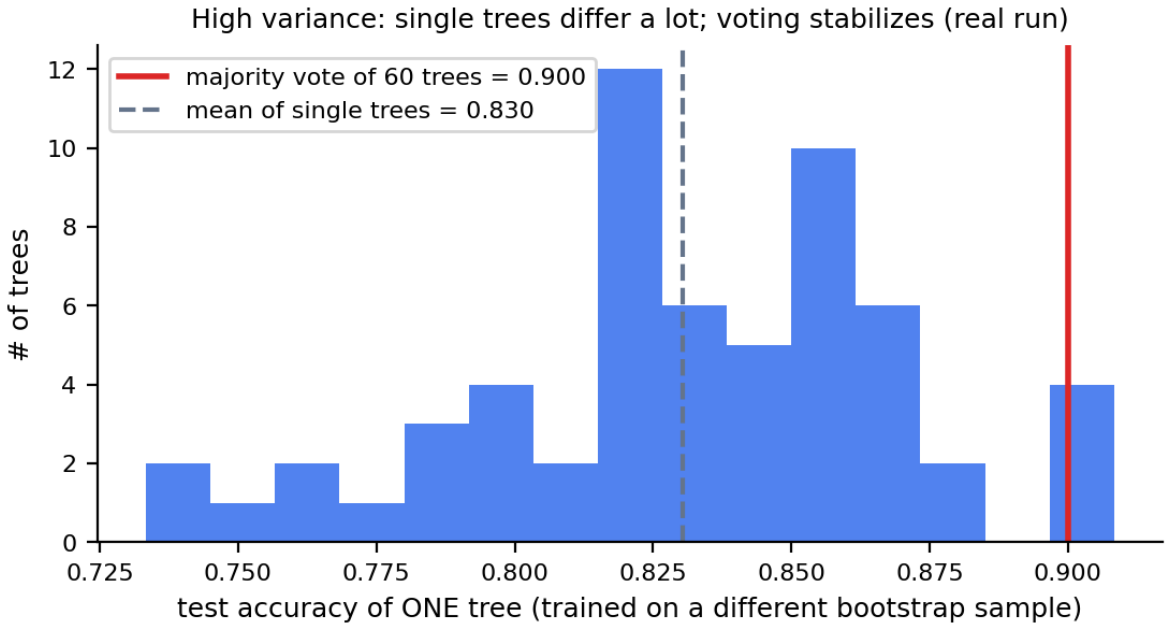
From your notebook

How do we reduce the effect of this problem? One of the important ways is to **aggregate the predictions of several trees** instead of relying on a single tree, in classification for example. This is the basic idea behind `Random Forest`, which we will study in the next chapter: aggregating trees may reduce the variance resulting from the sensitivity of a single tree.

```
rng = np.random.RandomState(5); preds, accs = [], []
for _ in range(60):
    idx = rng.randint(0, len(X_tr), len(X_tr))        # a bootstrap sample (random sample WITH
replacement)
    tree = DecisionTreeClassifier(random_state=0).fit(X_tr[idx], y_tr[idx])
    accs.append(tree.score(X_te, y_te)); preds.append(tree.predict(X_te))
vote = (np.mean(preds, axis=0) > 0.5).astype(int)     # majority vote of the 60 trees
print(np.mean(accs), np.std(accs), min(accs), max(accs)); print(accuracy_score(y_te, vote))
```

Output (real run)

```
0.830 0.039 0.733 0.908
0.900
```



A real experiment (moons, 60 trees, each one on a different bootstrap sample): the accuracy of a single tree ranges from 0.73 to 0.91 (mean 0.83) = high variance. But the vote of the 60 trees (the red line) = 0.90, better than almost any single tree. This is the idea of the Random Forest coming next.

14 Quick review questions

Answer first in your head, then check the answers on the next page.

1. What is a Decision Tree? And when is it used in Classification and when in Regression?
2. What is the meaning of `max_depth` and `fit` in the Iris code?
3. What is the difference between Root, Node and Leaf?
4. What is the meaning of samples, value and gini in each node?
5. Write the Gini equation. And what is its value for a pure node? And compute it for [0, 49, 5].
6. What are the Decision Boundaries? And what changes when we increase `max_depth`?
7. Why does the tree not need feature scaling?
8. What is the difference between White Box and Black Box? And does reading the tree guarantee it is fair?
9. How does the tree compute `predict_proba`? And what happens to two points in the same leaf?
10. What is the CART algorithm? And write its cost function for classification.
11. Why is CART called a Binary Tree? And what is the difference with ID3?
12. What does Greedy mean? And why does CART not try all the possible trees?
13. What is the computational complexity for prediction and for training? And when does prediction become slower?
14. What is Entropy? And the difference between it and Gini?
15. List 6 hyperparameters to reduce overfitting in the tree.
16. Why does a tree without restrictions overfit? And do the restrictions always raise the accuracy?
17. What is Pruning? And what is the role of the χ^2 test and the p-value?
18. What does each leaf output in a DecisionTreeRegressor?
19. Write the cost function of CART for regression.
20. Why is a Decision Tree sensitive to the orientation of the axes? And how can PCA help? And does it always help?
21. What is the meaning of the Decision Tree having high variance? And what is the relationship with `random_state`?
22. How do we reduce the variance? And what is the relationship of this with Random Forest?

15 Review question answers

1. A model that asks sequential questions about the features and every answer leads to a branch until a final decision. Classification: predicts a class. Regression: predicts a number (the mean of the target in the leaf).
2. `max_depth` = the maximum depth of the tree (to simplify it). `fit(X, y)` = training: the tree searches for the best questions that separate the types.
3. Root: the first node. Node: a node containing a decision/test. Leaf: the end of the branch and from it the prediction.
4. samples = the number of training samples that reached the node. value = the number of samples from each class. gini = the degree of mixing of the classes (0 = pure).
5. $G = 1 - \Sigma\, p^2$. For a pure node = 0. For [0, 49, 5]: $1 - (49/54)^2 - (5/54)^2 = 0.168$.
6. The regions the tree divides the space with (rectangles). Increasing `max_depth` = more questions = smaller regions and more complex boundaries.
7. Because each question compares one feature with a threshold (like petal length ≤ 2.45) and does not depend on distance or scale.
8. White Box: we can read the questions and branches that led to the decision. Black Box: interpretation is hard. But the ease of reading does not guarantee the decisions are correct or fair.
9. The proportion inside the leaf: samples of the class ÷ samples of the leaf. Any two points in the same leaf get the same probabilities.
10. CART = Classification and Regression Tree: at every node it chooses a feature and a threshold that minimize $J = (m_{left}/m)\cdot G_{left} + (m_{right}/m)\cdot G_{right}$.
11. Because every split gives only two branches. ID3 (in its traditional form) can produce more than two branches according to the values of the feature.
12. Greedy = it chooses the best split for the current node only. Finding the optimal tree is NP-complete (O(exp(m))) so it is practically impossible.
13. Prediction: $\approx O(\log_2 m)$ for a balanced tree (it depends on the actual depth, and is slower if the tree is unbalanced). Training: $\approx O(n \times m \log m)$.
14. Entropy = $-\Sigma\, p \log_2 p$, it measures the uncertainty (0 for a pure node). Gini is computationally simpler; both often give similar trees but not always the same shape.
15. `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_weight_fraction_leaf`, `max_leaf_nodes`, `max_features`.
16. Because it learns the details and noise of the training data. The restrictions do not always raise the accuracy, but they prevent some splits so the generalization improves (and increasing them more than necessary = underfitting).
17. Let the tree grow then delete the branches that do not improve the purity statistically significantly. χ^2 estimates whether the improvement is chance, and a p-value larger than a limit (5%) means deleting the node and its children. (sklearn uses `ccp_alpha` instead of it.)
18. Every leaf outputs the mean of the target values of the training samples that reached it.
19. $J = (m_{left}/m)\cdot MSE_{left} + (m_{right}/m)\cdot MSE_{right}$, where MSE_{node} is the mean of $(y - \hat{y}_{node})^2$ and $\hat{y}_{node}$ is the mean of the target in the node.

20. Because it makes boundaries perpendicular to the axes (horizontal/vertical), so it needs many splits to approximate a slanted line. PCA rotates the axes and one question may be enough, but it does not always improve (on Iris it did not make a difference).
21. That the tree changes a lot with small changes in the data or in the build choices (like breaking ties). `random_state` makes training reproducible but does not make the tree stable when the data changes.
22. By aggregating the predictions (voting) of many trees instead of a single tree, and this is the basis of the Random Forest in the next chapter.